The μZ Calculus and its Implementation

Wolfgang Grieskamp

Technische Universität Berlin, FB13, Institut für Kommunikations- und Softwaretechnik. E-mail: wg@cs.tu-berlin.de

Abstract. μZ is a small calculus which is designed as an intermediate language for the execution of a subset of the set-based specification language Z. In this paper, we present μZ's definition and computation model, and sketch its implementation. It is proposed that the provided model and implementation is of interest for the implementation of declarative languages in general.

1 Introduction

The automatic evaluation of *test-data* for *safety-critical embedded systems* is an interesting application which can help to bring declarative language implementation techniques into industrial practice. Some figures report about more then 50% of development costs in this application area which goes into testing. A setting for test-case evaluation which can improve this situation is as follows: given a requirements specification by an executable prototype, some input data describing a test-case, and the output data of a run of the system's implementation on the given input, by executing the specification we check whether the implementation meets its requirements. On the first sight, this goal seems to be simple, since input and output data are fixed. Yet, a real-world specification of a complex system may contain a lot of "hidden" data, which is used to describe the observable behavior. Thus the problem scales up to finding the solution(s) to a (sequence of) partial data bindings.

In the application-oriented research project ESPRESS[1], which is concerned with methods and tools for the development of embedded systems, the set-based specification language Z [16] is used for requirements specification, in combination with other notations such as Statecharts which are incorporated by a shallow encoding into Z [2]. Performing test-case evaluation in this setting requires a *computation model* for Z and its implementation.

In this paper, we report on some results of the efforts on implementing such a framework. The main theme of the paper is the presentation of the μZ calculus (Sect. 2), and the definition of a computation model for it (Sect. 3). The μZ calculus is a pure expression language with only a few constructs which abstracts from the diversity of Z. Since it can embed the full Z language – the simple-typed

[1] The ESPRESS project is a cooperation of industry (Daimler-Benz AG and Robert Bosch AG) and academic (GMD-First, FhG-ISST, and TU-Berlin) research institutes, funded by the German Bundesministerium für Bildung, Wissenschaft, Forschung und Technologie.

λ-calculus, set calculus, predicate calculus, and schema calculus – computation in μZ can indeed not be complete: but a computation model and implementation by an abstract machine can be provided which is comparable to that of functional logic paradigms (e.g. [5]).

The work presented in this paper constitutes the plan for refining an existing implementation of Z, the so-called ZAP compiler, which is part of the ESPRESS tool environment ZETA[4]. Within ZAP, Z is reduced to μZ, and μZ is compiled to Java, using a functional (deterministic) approach with concurrent constraint resolution for dealing with the problem of undirected equality and "parallel and". Several case studies performed together with industry partners demonstrated the feasibility of the principal approach, but also clearly showed that (1) deterministic computation for test-data evaluation is a serious restriction, since it constraints the way of how requirement specifications can be formulated in Z significantly (2) Java is a high performance penalty as the target for compiling a declarative language. Thus this work sets out to define an enhanced computation model for μZ, incorporating indeterministic logic principles, and designing an abstract machine which can be efficiently implemented in C++.

Using a set-based calculus to view computation might raise the impression that set-based resolution techniques shall be incorporated, such as set-unification [17], or subset clauses [12]. Yet, this is not the case: μZ and its implementation rather provides an alternative and unorthodox view on functional logic computation, where sets provide the framework for a smooth integration of these two concepts.

2 The μZ Calculus

In this section the definition of the μZ calculus is given: its syntax, semantics, and equational theory. As the meta language we loosely orient at the Z notation.

2.1 Syntax and Informal Semantics

Let $x \in VAR$ be variable symbols, and let $\pi \in CONS$ be (term) constructor symbols. The μZ calculus knows the syntactic categories of *expressions*, and a subset of them, *patterns*:

$$\begin{array}{ll} e \in EXP ::= & x \mid \pi(\overline{e}) \mid \{e\} \mid \mu\, e \mid \mathsf{0} \mid e \cap e' \mid e \cup e' \mid \sim e \mid \\ & \quad e(\!|p_1 \mapsto p_2|\!) \mid [\, p \mid e \,] \mid \mathbf{fix}\, p \lhd e \\ p \in PAT \subseteq & EXP \end{array}$$

The form $\pi(\overline{e})$ describes a constructor application, where $\overline{e}$ is a sequence of expressions. (We generally use the convention to denote sequences of terms t by $\overline{t}$.) The set of patterns $PAT \subseteq EXP$ are those expressions which are solely build from constructor application to patterns, $\pi(\overline{p})$, and variables, x.

The form $\{e\}$ denotes a singleton set. The form $\mu\, e$ is the singleton set selection, which returns the element of a singleton set, and is undefined for each other set. The usual forms for set algebra are provided: the empty set is denoted by $\mathsf{0}$, set intersection by $e \cap e'$, set union by $e \cup e'$, and set complement by $\sim e$.

The form $e(\!|p_1 \mapsto p_2|\!)$ describes the *translation* of the set e, and is not so familiar (it may found its roots in category theory). The result of translation consists of all elements which match against p_2, such that there exists an element in e which matches against the pattern p_1 under the same substitution. Thereby, both patterns may contain unbound variables, which may be denoted by wildcards ($_$). For example, the cartesian product of two sets, $e_1 \times e_2$, can be expressed in μZ as $e_1(\!|x \mapsto (x, _)|\!) \cap e_2(\!|y \mapsto (_, y)|\!)$ (with $(_, _)$ the constructor symbol for pairs used in mixfix notation). Vice versa, the range of a binary relation e (a set of pairs) is denoted as $e(\!|(_, y) \mapsto y|\!)$. We call a translation with $\mathsf{vars}\, p_1 \subseteq \mathsf{vars}\, p_2$ an *embedding* and with $\mathsf{vars}\, p_1 \supset \mathsf{vars}\, p_2$ a *hiding*. Note that there might exist translations which are neither embeddings or hidings.

The form $[\, p \mid e \,]$ describes a set-comprehension, or a *schema* in Z speech. It denotes the set of values which match the pattern p such that the boolean expression e is true under the binding of variables resulting by the match. A boolean expression is thereby just a set-expression over the element type of units (a type containing just one element): let $\diamond$ be the 0-ary constructor for this element, then truth is the set $\{\diamond\}$, falsity the empty set, conjunction intersection, disjunction union, and negation complement. For example, the (sugared) schema $[\, (x, y, z) \mid x < y \cap y < z \,]$ describes the set of triples where each member is larger then the previous one. The boolean expression $x < y$ thereby is syntactic sugar for the form $((_ < _) \cap \{(x, y)\})(\!|_ \mapsto \diamond|\!)$ (where $_ < _$ is a binary relation, a set of pairs). The conclusive translation can be read as: we are not interested in the actual result of the intersection, we just want to know if it is empty or not.

The form $\mathbf{fix}\, p \triangleleft e$ describes the smallest member of the set $[\, p \mid p = e \,]$ (where the equality abbreviates $(\{p\} \cap \{e\})(\!|_ \mapsto \diamond|\!)$). The order needed here is discussed in Sect. 2.3. If a smallest member does not exists, the form is undefined.

The μZ calculus uses a shallow-polymorphic type system which, since it is fairly standard, is not made explicit within this paper. To this end, constructors π have associated a (generic) type, and expressions are assumed to have a principal type. A further condition for well-formedness is that patterns in translations and schemas are used linear (no variable is allowed to appear twice).

For commonly used forms of expressions, syntactic sugar is used:

$$
\begin{array}{lll@{\qquad}lll}
1 & \rightsquigarrow \sim 0 & & \bigcup\langle\rangle & \rightsquigarrow 0 \\
?_1\, e & \rightsquigarrow e(\!|x \mapsto \diamond|\!) & & \bigcup(\langle e\rangle \frown \overline{e}) & \rightsquigarrow e \cup \bigcup \overline{e} \\
e \in e' & \rightsquigarrow ?_1(\{e\} \cap e') & & \bigcap\langle\rangle & \rightsquigarrow 1 \\
e = e' & \rightsquigarrow e \in \{e'\} & & \bigcap(\langle e\rangle \frown \overline{e}) & \rightsquigarrow e \cap \bigcap \overline{e}
\end{array}
$$

Following Z conventions, $\langle a_1, \ldots, a_n\rangle$ is a sequence display, and $_ \frown _$ sequence concatenation.

2.2 Mappings

Before we take a closer look on the meaning of μZ expressions, the translation of Z into μZ is discussed. A brief comparison with the functional logic language Curry [7] is also given, anticipating some basics about the computation model of μZ.

Z. The operations of the simple-typed λ-calculus contained in Z are mapped as follows:

$$\begin{array}{lll} \lambda\, p \bullet e & \leadsto & [\,(p, y) \mid y = e\,] \qquad\qquad (y \notin \mathit{vars}\, e) \\ e\, e' & \leadsto & \mu((e \cap \{e'\}\,\{\!|x \mapsto (x, _)|\!\})\,\{\!|(_, y) \mapsto y|\!\}) \end{array}$$

This handles functions as sets of pairs. The mapping of the application in fact also captures relational application as used in Z: it is sufficient that e is a binary relation which is right-unique and defined just at the point e'.

We have already noted that predicate expressions are a special case of set-expressions over the element type of units, and have defined abbreviations for the basic predicates of membership-test and equality. To make the picture complete, here is the mapping of quantifiers:

$$\exists\, x \bullet e \;\leadsto\; ?_1[\,x \mid e\,] \qquad\qquad \forall\, x \bullet e \;\leadsto\; \sim(?_1[\,x \mid \sim e\,])$$

The underlying semantic of the mapping of predicate calculus is a partial three-point logic, as discussed in Sect. 2.3.

In Z, a schema denotes a set of *bindings*, where bindings are tuples (records) with named components. The set is constraint by the declarations of a schema as well as by the property. For example, the Z schema $[a : \mathbb{N};\ b : \mathbb{Z} \mid a < b]$ denotes the set of bindings $\langle\!|a \Rrightarrow x, b \Rrightarrow y|\!\rangle$, where $x \in \mathbb{N}$, $y \in \mathbb{Z}$, and $x < y$. Assuming appropriate μZ constructors for bindings, the above Z schema is mapped to μZ by $\mathbb{N}\{\!|x \mapsto \langle\!|a \Rrightarrow x, b \Rrightarrow _|\!\rangle|\!\} \cap [\,\langle\!|a \Rrightarrow x, b \Rrightarrow y|\!\rangle \mid x < y\,]$ (the constraint $y \in \mathbb{Z}$ vanishes since $\mathbb{Z}$ is a given type). The property $x < y$ is further translated as described in the previous section.

The connectivities of the schema calculus easily map as well. Schema conjunction in Z, $S_1 \wedge S_2$, is the set of bindings with the joined signature of S_1 and S_2 where components with the same name are identified. This is expressed in μZ as $S_1\{\!|\Sigma(S_1) \mapsto \Sigma|\!\} \cap S_2\{\!|\Sigma(S_2) \mapsto \Sigma|\!\}$, where Σ is the pattern for the joined signature, and the same pattern variables are chosen at the identified positions. Existential schema quantification, $\exists\, S \bullet S'$, is mapped to $(S\{\!|\Sigma(S) \mapsto \Sigma(S')|\!\} \cap S')\{\!|\Sigma(S') \mapsto \Sigma(S') \setminus \Sigma(S)|\!\}$ (where $\Sigma(S') \setminus \Sigma(S)$ denotes the removal of the components from S in S'. Similarly, the remaining connectivities of the schema calculus are mapped.

Functional Logic Languages. Languages such as Curry[7] can be mapped in principle to μZ. The exact operational semantics of such a mapping will not be the same, since for example in μZ a strict reduction order is preferred by design. An illustration of such a mapping is helpful anyway.

A function definition in Curry with constraints is mapped as follows (where the fresh variables y_i below must not appear in the patterns p_i):

$$\begin{array}{l} f\, p_1 \mid c_1 \;= e_1;\; \ldots\; ;\, f\, p_n \mid c_n \;= e_n \;\leadsto \\ \qquad \mathbf{fix}\, f \lhd [\,(p_1, y_1) \mid c_1 \cap y_1 = e_1\,] \cup \ldots \cup [\,(p_n, y_n) \mid c_1 \cap y_n = e_n\,] \end{array}$$

Thereby, an equational constraint c of Curry directly maps to an according equation in μZ. Conjunctions of constraints are expressed by intersection. Calling a function which returns a constraint with a free variable as the argument,

such as $C\,x$, amounts to membership test, $x \in C$. In fact, μZ allows a richer constraint language than Curry (for example, disjunction and universal quantification) which, however, will not always be executable.

The computation model of μZ as described later on executes constraints concurrently, using both residuation as well as backtracking to deal with free variables. An access to a free variable is generally suspended in a nested context. This is for example the case for arguments of the μ-operator, (and therefore for function application $e\,e'$ which is defined using μ), or for negative constraints. Backtracking may take place for constraints which map to the membership test of a pattern in a set. For illustration consider the definition of an "append" relation on lists. Below a shortcut for introducing local variables is used: $[\,p \setminus q \mid e\,]$ is equivalent to $[\,(p,q) \mid e\,]\{\!|(p,q) \mapsto p|\!\}$:

$$\mathbf{fix}\ app \lhd \begin{array}{l}[\,(\langle\rangle, ys, zs) \mid zs = ys\,] \cup \\ [\,(x :: xs, ys, zs) \setminus t \mid (xs, ys, t) \in app \cap zs = x :: t\,]\end{array}$$

Applying this relation to unbound xs and ys but bound zs will compute the possible partitions of the list zs, as usual in logic languages.

μZ is *higher-order* since sets can be stored in variables – and the relation *app* as defined above is nothing more than a set. Yet, as Curry, the μZ's computation model does not employs *higher-order unification*. This restriction is, however, not bounded to the notion of a type of a value (whether it is a set or not) but to the representation: sets which are *extensionally* represented are incorporated by a simple equality test in unification. An extensional representation is given by applying constructors, singleton sets, and set union to extensional values.

2.3 Semantics

We will not give a detailed semantic mapping here, but only discuss the principles. The model of μZ is based on a hierarchical typed universe, where sets are represented as *partial characteristic (boolean) functions* in a set-theoretical meaning, which can be founded by Z itself or by ZF set theory [18]. This representation of sets is a generalization of partial three-point logics [13] to the case of set-algebra. There are three observation to be made about a set represented as a partial characteristic function: an element is member (it is in the domain and is mapped to true), an element is not member (it is in the domain and is mapped to false), or it is unknown to be member or not (it is not in the domain). As a result, we are able to give expressions of the kind $[\,x \mid 1 \,\mathsf{div}\, x = 1\,]$ a meaning: for 0, membership is unknown, for all other numbers it is known (1 is member and the other numbers are not).

Indeed, for infinite sets the test whether an element is in the domain of a characteristic function is not computable, but it is possible to delay the need for this computation. For example, the intersection of two sets represented by partial characteristic functions is defined as $f_1 \cap f_2 = \lambda\, x \bullet f_1\, x \wedge f_2\, x$, where the conjunction can be executed by "parallel and". Thus the meaning behaves compositionally: $[\,x \mid 1 \,\mathsf{div}\, x = 1\,] \cap [\,x \mid x \neq 0\,]$ is a set with no unknowns. This is a prerequisite for a clean equational theory of μZ. The valid laws will

be discussed in more detail in the next section. In particular, we will see that De-Morgan's laws hold, such that a disjunctive normal form can be constructed.

To give the fixed-point operator of μZ a meaning, an order on values need to be defined. Since we have union and intersection as total functions on sets, a μZ set constitutes a *complete lattice*, with the empty set (the total characteristic function constantly delivering false) the bottom element and the universal set of the according type the top element. The order for fixed-point construction is derived from this: $f_1 \sqsubseteq f_2 \Leftrightarrow f_1 \cap f_2 = f_1$. The order is pointwise lifted to values yielded from constructor application. Since we are not in a constructive world *a priori*, $\mathbf{fix}\, p \triangleleft e$ might be not defined, since e may not be monotonic.

2.4 Equational Theory

μZ has a rich equational theory which can be used for partial evaluation and even for providing a narrowing semantics by plain expression transformation. Within this section, we will only discuss those laws which are necessary for establishing a disjunctive normal form as needed for the computation model.

The set operators of μZ follow the usual laws for *boolean algebras*. Commutativity and distributivity of union and intersection hold, and De Morgan's laws are satisfied such that we can build a conjunctive or disjunctive normal form. However, because of partiality, the principle of excluded middle does not hold ($\sim e \cap e \equiv \mathsf{0}$ does *not* hold). Yet, we have $\sim(\sim e) \equiv e$, and excluded middle for the case of singleton sets: $\{e\} \cap \sim\{e\} \equiv \mathsf{0}$.

A singleton set containing a constructor application with arity greater zero can be decomposed into an intersection of embeddings as follows:

$$\{\pi(e_1, \ldots, e_n)\} \equiv \{e_1\}(\!|x_1 \mapsto \pi(x_1, \ldots, _)|\!) \cap \ldots \cap \{e_n\}(\!|x_n \mapsto \pi(_, \ldots, x_n)|\!)$$

Several laws regarding set translation are of interest. Consecutive translations are composable, if the intermediate patterns can be unified. If they cannot be unified, consecutive translation reduces to the empty set. Any translation distributes over set union. A translation by an embedding distributes over set intersection. A translation which is neither hiding or embedding can be split into an embeddings and a consecutive hiding. Any hiding can be lifted out of an intersection, by widening the scope of the hidden variables. The last two laws read as follows:

$$\begin{array}{ll} e(\!|p_1 \mapsto p_2|\!) & \equiv\ e(\!|p_1 \mapsto (\,\overline{x}\,)|\!)\,(\!|(\,\overline{x}\,) \mapsto p_2|\!) \\ \quad , \mathbf{if}\ \neg\ \mathsf{vars}\ p_1 \subseteq \mathsf{vars}\ p_2;\ \neg\ \mathsf{vars}\ p_1 \supseteq \mathsf{vars}\ p_2;\ \mathsf{ran}\,\overline{x} = \mathsf{vars}\ p_1 \cup \mathsf{vars}\ p_2 & \\ e(\!|p_1 \mapsto p_2|\!) \cap e' \equiv (e \cap e'(\!|p_2 \mapsto p_1|\!)\,)(\!|p_1 \mapsto p_2|\!) & , \mathbf{if}\ \mathsf{vars}\ p_1 \supset \mathsf{vars}\ p_2 \end{array}$$

Hence, since embeddings can be distributed and hidings can be lifted out of intersections, meanwhile other translations can be decomposed into embeddings and hidings, it is possible to normalize expressions such that embeddings are pushed to the leafs and hidings are pulled to the top of intersections.

Unfortunately, set complement does not distribute over translation, and translation cannot be lifted out of a complement in the general case. It is notable in this context that the complement of a hiding is related to a $\forall$-quantor: $\forall\, x \bullet e$

maps to $\sim([\,x \mid \sim e\,](\!|x \mapsto \diamond|\!))$. The fact that complement cannot be pushed to the leaves is a hint to the necessary incompleteness of any computation model.

Schema construction distributes over union and intersection: $[\,p \mid e \cup e'\,] \equiv [\,p \mid e\,] \cup [\,p \mid e'\,]$, and similarly for intersection. Set complement distributes over schema construction as follows: $\sim[\,p \mid e\,] \equiv [\,p \mid \sim e\,] \cup \sim(\mathsf{1}(\!|p \mapsto p|\!))$. Those elements need to be added which are in the complemented schema's meaning since they do not match the pattern p.

3 A Strict Computation Model for μZ

A strict computation model for μZ is described in the style of natural semantics [10]. The intention is to get a model which is abstract enough to provide an idea of computation in μZ, but also near enough to an implementation to serve as a starting point for its design.

The computation model is not primarily designed with the goal to get "maximal" computation power out of μZ. A complete model cannot be realized anyway. Efficient implementability and transparent traceability of execution are as significant as computation power. The last point is in particular important for the application to test-case evaluation, where the reason of the failure of a test needs to be understandable by humans. A major design decision derived from these considerations is to use a *strict* evaluation order. Strictness in the context of μZ means that members of the singleton set, $\{e\}$, are completely evaluated before the set is constructed. Similarly, in constructed values, $\pi(\overline{e})$, the arguments are fully evaluated prior to the constructor call. Moreover, we will not provide "parallel or" ($\bot \vee \mathit{true} \equiv \mathit{true} \vee \bot \equiv \mathit{true}$): disjunctions are computed from left to right[2]. However, "parallel and" ($\bot \wedge \mathit{false} \equiv \mathit{false} \wedge \bot \equiv \mathit{false}$) is provided in non-strict contexts, i.e. the properties of schemas. Thus $[\,p_1 \mid e_1\,] \cap [\,p_2 \mid e_2\,]$ is $\mathsf{0}$ if one of the properties e_i is false, independent of possible divergency of the other property, whereas $e_1 \cap [\,p \mid e_2\,]$ will be always undefined if e_1 is undefined.

3.1 Disjunctive Normal Form

An expression inside a complement will not contribute to resolution in its context. Thus it makes a difference whether distributivity laws for pushing a complement into expressions have been applied. Moreover, only certain forms of properties of schemas contribute to resolution. Hence it is necessary to normalize expressions prior to constructing their operational semantics. To this end, the *disjunctive normal form* of expression is defined as follows:

$$
\begin{array}{lll}
b \in EXP_B & ::= d \mid \pi(\overline{b}) \mid \mu\, d \mid x & \textit{any} \\
d \in EXP_D & ::= \bigcup \overline{c} & \textit{disjunct.} \\
c \in EXP_C & ::= \bigcap \overline{l} & \textit{conjunct.} \\
l \in EXP_L & ::= \sim a \mid a & \textit{literal} \\
a \in EXP_A & ::= \mathsf{0} \mid x \mid [\,p \mid \phi\,] \mid c(\!|p_1 \mapsto p_2|\!) \mid \mu\, d \mid \mathbf{fix}\, p \triangleleft d \mid \{b\} & \textit{atom} \\
\phi \in EXP_P & ::= p \in d \mid p_1 = p_2 \mid ?_0\, d \mid ?_1\, d & \textit{property}
\end{array}
$$

[2] The computation model can be easily modified to relax this condition, but an implementation providing "parallel or" is signifcantly more complex.

The normal form is justified by the de-Morgan laws of the calculus, as well as the several distributivity laws for translation. Beneath a translation there might still appear a conjunction, since translations cannot be pulled out of complements. We assume that complement is pushed as far as possible. Hidings should be pulled, embeddings pushed, and consecutive translations contracted. A lot of further partial evaluation which follows from the equational laws could be applied on normalization; however, apart of complement pushing, none of these really affects the power of the computation model.

The representation of schema properties, ϕ, determines the contribution to resolution. The form $?_1\, d$ takes a set and tests if it is the non-empty set, the form $?_0\, d$ is a shortcut for $\sim ?_1\, d$ and tests if its operand is the empty set. These properties are executed using residuation, thus all variables in d need to be bounded before the test can be completed. The form $p \in d$ denotes the member test of a pattern p in the disjunction d, where d is reduced by residuation. The form $p_1 = p_2$ demands the unification of two patterns. The variables appearing in the patterns p, p_1, and p_2 must be directly bounded by the enclosing schema.

It can be shown that any *flexible* appearances of variables in a property can be converted into a membership-property together with a sequence of simple equations of flexible variables, $x = x'$. By a flexible appearance we mean a variable x used as in $[\, P\, x \mid C\, \{\Pi\, x\}\,]$, where C is a positive conjunctive context build from intersections and translations, and Π a context of constructor applications.

Proof. Let $[\, p \mid ?_1(\bigcap \overline{l})\,]$ be a schema with a positive conjunctive property as an intermediate result of normalization, where constructor decomposition has been performed, hidings have been pulled and absorbed by the overall hiding of $?_1_$, and embeddings have been pushed to the leaves. Henceforth $?_1(\bigcap \overline{l})$ must match (modulo associativity and commutativity)

$$?_1(\{x_1\}(\!|p_1 \mapsto q_1|\!) \cap \ldots \cap \{x_n\}(\!|p_n \mapsto q_n|\!) \cap e)$$

where the x_i are flexible and where e does not contains variables in flexible position. Next we unify the patterns q_i against each other (if this fails then the property reduces to $\mathsf{0}$ already at normalization time). Let γ be the resulting substitution, and let $\sigma = \{x_1 \mapsto \mathsf{subs}\,\gamma\ p_1, \ldots, x_n \mapsto \mathsf{subs}\,\gamma\ p_n\}$ be a substitution for the flexible variables x_i, and $q = \mathsf{subs}\,\gamma\ q_1$ the unified translation target. Let $\overline{x} = \mathsf{vars}\ q \cap \bigcup(\mathsf{vars}(\!|\mathsf{ran}\,\sigma|\!))$ the variables in q which are tight to the flexible variables (q can contain further variables introduced by embeddings). Then we construct the schema $[\, \mathsf{subs}\,\sigma\ p \mid (\overline{x}) \in (\mathsf{subs}\,\sigma\ e)(\!|q \mapsto (\overline{x})|\!)\,]$. Yet, this schema might not be well-formed, since $\mathsf{subs}\,\sigma\ p$ may result in a non-linear pattern. In this case, a simple renaming of double occurrences of variables is performed, adding aliases like $x = x'$ for each renamed variable. □

As an example consider the schema $[\, x \mid (e, x) \in e'\,]$. This results in the normalized schema

$$[\, x \mid x \in (\{e\}(\!|x \mapsto (x, _)|\!) \cap e')(\!|(_, y) \mapsto y|\!)\,].$$

The next example, $[\, (x, y, z) \mid (e, x) = (y, z) \in e'\,]$, demonstrates aliasing:

$$?_1(\{x\}(\!|x \mapsto (_, x)|\!) \cap \{y\}(\!|x \mapsto (x, _)|\!) \cap \{z\}(\!|x \mapsto (_, z)|\!) \cap \{e\}(\!|x \mapsto (x, _)|\!) \cap e')$$

Unification of $(_, x)$ with $(y, _)$ and $(_, z)$ yields (y, z). The resulting schema is

$$[\,(z, y, z') \mid (y, z) \in (\{e\}(\!|x \mapsto (x, _)|\!) \cap e') \cap z = z'\,].$$

In a subsequent normalization step this is reduced to a conjunction of two schemas, one holding the pattern-membership, the other the alias.

3.2 Values and Constraints

Values are a subset of normalized expressions, enriched by a tailored representation of a schema-like construct, called an *intension*. An intension consists of a pattern and a *constraint*. A basic constraint, $\sigma \parallel \phi$, is a value assignment paired with a property expression. Constraints may be composed by conjunction and by disjunction, though we expect only conjunctions in intensions (disjunctions are needed later on):

$$\begin{array}{lll} v \in EXP_V & ::= \pi(\overline{v}) \mid \{v\} \mid [\![p \mid \theta]\!] \mid \bigcup \overline{v} \\ \theta \in CTR & ::= \sigma \parallel \phi \mid \theta_1 \wedge \theta_2 \mid \theta_1 \vee \theta_2 \\ \sigma \in VALASS & == VAR \twoheadrightarrow EXP_V \end{array}$$

An intension $[\![p \mid \sigma \parallel \phi]\!]$ can be interpreted as the schema $[\,p \mid \mathsf{subs}\,\sigma\,\phi\,]$. However, there are a few differences. First, it is possible that the above expansion results in an infinite term, since σ may contain an assignment referring to the intension itself. This circularity of assignments is used to express fixed-points in the computation model. Second, the property ϕ may contain free variables which are neither bounded by p not by σ. These variables may be considered as "local existential" for the intension, and are used to represent hiding translations by intensions.

A (partial) equality on values, written as $v \sim v'$, is assumed. The equality takes commutativity and associativity of unions into account. Since equality of intensions cannot be decided, a second relation $v \not\sim v'$ is required, which decides inequality. On base of value equality and inequality, a unification for "extended patterns" build from constructor application, variables, and values, written as $p_1 \sim^\sigma p_2$, is assumed. This unification uses $_\sim_$ for comparing values found at the bottom of constructor terms. The form $p_1 \not\sim^\sigma p_2$ decides whether unification fails (using $_\not\sim_$).

In principle, more powerful notions of unification can be embedded in our framework (e.g. set unification). This would require some technical modifications of the rules given in the next section to deal with *sets* of unifiers returned by unification instead of a single unifier.

3.3 Computation

Computation is defined by two relations, which are mutually dependent. The first relation, $e \xrightarrow{\sigma} e'$, describes a strict reduction step on expressions under the value assignment σ. It cannot be applied to unbound variables, which means in the context of its usage that the according computation "suspends". The second relation, $\theta \rightsquigarrow \theta'$, describes a resolution step by mapping a constraint into a constraint. θ is expected to be in disjunctive normal form, and θ' will be so as well.

Fig. 1 Expression Reduction Rules

$$E1\ \frac{e \xrightarrow{\sigma} e'}{STRICT\ e \xrightarrow{\sigma} STRICT\ e'} \qquad E2\ \frac{x \in \mathsf{dom}\,\sigma}{x \xrightarrow{\sigma} \sigma\, x} \qquad E3\ \frac{\mathsf{vars}\ e \subseteq \mathsf{vars}\ p \cup \mathsf{dom}\,\sigma}{[\,p \mid e\,] \xrightarrow{\sigma} [\![p \mid \sigma \parallel e]\!]}$$

$$E4\ \frac{p \sim^{\sigma'}_{\sigma} v;\ e \xrightarrow{\sigma'} e_1 \xrightarrow{\sigma'} \ldots e_n \xrightarrow{\sigma'} v}{\mathbf{fix}\, p \lhd e \xrightarrow{\sigma} v}$$

$$E5\ \frac{}{\bigcup V\langle 0\rangle \xrightarrow{\sigma} \bigcup V\langle\rangle} \qquad E6\ \frac{v_1 \sim v_2}{\bigcup V\langle v_1, v_2\rangle \xrightarrow{\sigma} \bigcup V\langle v_1\rangle}$$

$$E7\ \frac{}{\bigcap V\langle 0\rangle \xrightarrow{\sigma} 0} \qquad E8\ \frac{v_1 \not\sim v_2}{\bigcap V\langle v_1, v_2\rangle \xrightarrow{\sigma} 0} \qquad E9\ \frac{v_1 \sim v_2}{\bigcap V\langle v_1, v_2\rangle \xrightarrow{\sigma} \bigcap V\langle v_1\rangle}$$

$$E10\ \frac{p_1 \sim^{\sigma'} p_2;\ exvs_2 = \mathsf{vars}\,\theta_2 \setminus \mathsf{vars}\ p_2;\ \gamma \in exvs_2 \rightarrowtail fresh}{\bigcap V\langle [\![p_1 \mid \theta_1]\!], [\![p_2 \mid \theta_2]\!]\rangle \xrightarrow{\sigma} \bigcap V\langle [\![\mathsf{subs}\,\sigma'\ p_1 \mid \mathsf{subs}\,\sigma' \rhd (\theta_1 \wedge (\gamma \rhd \theta_2))]\!]\rangle}$$

$$E11\ \frac{p_1 \not\sim^{\sigma'} p_2}{\bigcap V\langle [\![p_1 \mid \theta_1]\!], [\![p_2 \mid \theta_2]\!]\rangle \xrightarrow{\sigma} 0} \qquad E12\ \frac{}{\bigcap V\langle v_1 \cup v_2\rangle \xrightarrow{\sigma} \bigcap V\langle v_1\rangle \cup \bigcap V\langle v_2\rangle}$$

$$E13\ \frac{}{v(\!|p_1 \mapsto p_2|\!) \xrightarrow{\sigma} [\![p_2 \mid \varnothing \parallel p_1 \in v]\!]} \qquad E14\ \frac{x \in fresh}{\sim\ v \xrightarrow{\sigma} [\![x \mid \varnothing \parallel ?_0(x \in v)]\!]}$$

$$E15\ \frac{x \in fresh}{\mu\ v \xrightarrow{\sigma} \mu_{\mathrm{E}}(x, \varnothing \parallel x \in v)} \qquad E16\ \frac{\theta \rightsquigarrow \theta'}{\mu_{\mathrm{E}}(x, \theta) \xrightarrow{\sigma} \mu_{\mathrm{E}}(x, \theta')}$$

$$E17\ \frac{\#\,\overline{\sigma} > 0;\ \forall\, i : \mathsf{dom}\,\overline{\sigma} \bullet x \in \mathsf{dom}(\overline{\sigma}\ i);\ \forall\, i, j : \mathsf{dom}\,\overline{\sigma} \bullet \overline{\sigma}\ i\, x \sim \overline{\sigma}\ j\, x}{\mu_{\mathrm{E}}(x, \bigvee(\overline{\sigma} \parallel \overline{1})) \xrightarrow{\sigma} \overline{\sigma}\ 1\, x}$$

A few notational conventions for defining these relations are introduced. We write $\bigwedge \overline{\theta}$ for a conjunction of constraints, and $\bigvee \overline{\theta}$ for a disjunction. For a sequence of basic constraints, $\overline{\sigma} \parallel \overline{\theta}$ is written. The set of free variables used in a constraint can be retrieved by $\mathsf{vars}\,\theta$. The notation $p_1 \sim^{\sigma'}_{\sigma} p_2$ "refines" the assignment σ to the assignment σ' by the result of unifying p_1 with p_2. With $\sigma \rhd \theta$ the *propagation* of an assignment into a constraint is described, which distributes over conjunction and disjunction. For basic constraints we have $\sigma \rhd (\sigma' \parallel \phi) = (\sigma' \circ \sigma) \parallel \phi$, where $_ \circ _$ is the usual composition of mappings. The common assignment of a conjunction of constraints is extracted with $\sigma \lhd \theta$: for basic constraints we have $\sigma \lhd \sigma \parallel \phi$, and for conjuncted ones $\sigma \lhd (\theta_1 \wedge \theta_2)$ iff $\sigma = \sigma_1 \cup \sigma_2 \in VALASS$ and $\sigma_i \lhd \theta_i$ $(i \in \{1, 2\})$.

The rules for $e \xrightarrow{\sigma} e'$ are given in Fig. 1. Rule $E1$ describes a reduction step in an arbitrary strict expression context $STRICT$, which is any context apart of the property of a schema. Rule $E2$ describes the substitution of a variable, which will be only possible if the variable is bounded in σ. This is one source of "suspension". Rule $E3$ reduces a schema to an intension. The rule is only applicable if free variables of the property are either bounded by the pattern or by the substitution – that is variables from outer scopes are not allowed to be free. This is the other source of "suspension". Rule $E4$ describes the construction of a fixed-point. The rule is cyclic in its premise, forestalling the resulting value v

in an assignment σ' which is used to calculate this value (the usual technique used in natural semantics for constructing fixed-points). A finite number of reduction steps is required to map the expression e to a value v: the expression e may contain several schemas, and the value form is reached not before all these schemas have been converted to intensions.

Rules $E5$ to $E6$ describe the elimination of tautologies in set union of values, rules $E7$ to $E9$ similar for set intersection of values. With $V\langle v_1, \ldots, v_n\rangle$ a value sequence context is denoted, where the v_i appear in some order in the sequence. Rules $E10$ and $E11$ model the combination of intensional values by intersection. If the patterns can be unified, then we simply concatenate the constraints of the intensions, making local existential variables disjunct; if they do not unify, the intersection reduces to $\mathsf{0}$. Rule $E12$ describes the distribution of intersection over union values. It should be noted that in an implementation intersection can be realized more efficiently as hinted by the rules: given a total order on values, and set values in a normalized form $\bigcup\{\overline{v}\} \cup \bigcup[\![\overline{p} \mid \overline{\theta}]\!]$, their intersection can be realized by processing the extensional part, $\overline{v}$, in linear time, and then the combinations with the intensional part.

Rule $E13$ handles the reduction of a translation to an intension. It uses the possibility to use existential variables in an intensions constraint: p_1 may contain variables not bounded by p_2. Rule $E14$ describes the translation of complement to an intension.

Rules $E15$ to $E17$ handle the μ-operator. Once the argument of μ has been reduced to a value v, the elements of v are enumerated by resolving a constraint $x \in v$. The state of this sub-resolution is stored in an intermediate expression construct, $\mu_{\mathrm{E}}(x, \theta)$. Once the sub-resolution is in solved form, $\bigvee(\overline{\sigma} \parallel \overline{1})$, a unique assignment for x is extracted, if it exists.

The rules for constraint resolution, $\theta \rightsquigarrow \theta'$, are given in Fig. 2. Rule $C1$ describes the left-to-right resolution of disjunctive constraints, rule $C2$ the elimination of unsolvable constraints. Rules $C3$ and $C4$ describe the elimination of unsolvable and solved constraints in a conjunction, where $S\langle\theta\rangle$ is a sequence context. Rule $C5$ describes a resolution step of a basic constraint in context of a conjunction. This rule models the concurrent execution of constraints by "interleaving", since the selection of a constraint in the context S is undetermined. The selected constraint rewrites to a disjunction. The conjunction context S is expanded for each member of this disjunction. In an implementation, this expansion can be implemented by backtracking (Sect. 4).

Rules $C6$ to $C7$ handle the reduction of expressions to values in basic constraints by $e \xrightarrow{\sigma} e'$. Rule $C8$ decomposes the membership test on a value union into disjuncted constraints. Rule $C9$ and $C10$ describe a basic resolution step, reducing a membership-test on a singleton set to an assignment or a failure. Rule $C11$ and $C12$ describe the resolution of a member-ship test on an intension. The constraint of the intension is imported into the current resolution. Variables used in the intension's pattern and constraint are renamed by γ to fresh ones. Finally, rules $C13$ and $C14$ model the resolution of a pattern equality.

Fig. 2 Constraint Resolution Rules

$$C1\ \frac{\theta \rightsquigarrow \bigvee \overline{\theta}'}{\bigvee(\overline{\sigma} \parallel 1 \frown \langle\theta\rangle \frown \overline{\theta}) \rightsquigarrow \bigvee(\overline{\sigma} \parallel 1 \frown \overline{\theta}' \frown \overline{\theta})} \qquad C2\ \frac{}{\bigvee(\langle\sigma \parallel 0\rangle \frown \overline{\theta}) \rightsquigarrow \bigvee \overline{\theta}}$$

$$C3\ \frac{}{\bigwedge(S\langle\sigma \parallel 0\rangle) \rightsquigarrow \sigma \parallel 0} \qquad C4\ \frac{}{\bigwedge(S\langle\sigma \parallel 1\rangle) \rightsquigarrow \bigwedge(S\langle\rangle)}$$

$$C5\ \frac{\theta \rightsquigarrow \bigvee \overline{\theta}';\ n = \#\overline{\theta}' = \#\,\overline{\sigma};\ \forall\, i : 1\,..\,n \bullet \overline{\sigma}\ i \lhd \overline{\theta}'\ i}{\bigwedge(S\langle\theta\rangle) \rightsquigarrow \bigvee\langle\overline{\sigma}\ 1 \rhd \bigwedge(S\langle\overline{\theta}' 1\rangle), \ldots, \overline{\sigma}\ n \rhd \bigwedge(S\langle\overline{\theta}'\ n\rangle)\rangle}$$

$$C6\ \frac{e \xrightarrow{\sigma} e'}{\sigma \parallel p \in e \rightsquigarrow \sigma \parallel p \in e'} \qquad C7\ \frac{e \xrightarrow{\sigma} e'}{\sigma \parallel ?_s e \rightsquigarrow \sigma \parallel ?_s e'}$$

$$C8\ \frac{}{\sigma \parallel p \in (v_1 \cup v_2) \ \rightsquigarrow\ \sigma \parallel p \in v_1 \ \vee\ \sigma \parallel p \in v_2}$$

$$C9\ \frac{p \sim_\sigma^{\sigma'} v}{\sigma \parallel p \in \{v\} \rightsquigarrow \sigma' \parallel 1} \qquad C10\ \frac{p \not\sim_\sigma^{\sigma'} v}{\sigma \parallel p \in \{v\} \rightsquigarrow \sigma \parallel 0}$$

$$C11\ \frac{\gamma \in \mathsf{vars}\,\theta \bigcup \mathsf{vars}\,p' \rightarrowtail \mathit{fresh} \quad p \sim_\sigma^{\sigma'} \mathsf{subs}\,\gamma\ p'}{\sigma \parallel p \in [\![p' \mid \theta]\!] \rightsquigarrow \sigma' \rhd (\gamma \rhd \theta)} \qquad C12\ \frac{\gamma \in \mathsf{vars}\,p' \rightarrowtail \mathit{fresh};\ p \not\sim_\sigma^{\sigma'} \mathsf{subs}\,\gamma\ p'}{\sigma \parallel p \in [\![p' \mid \theta]\!] \rightsquigarrow \sigma \parallel 0}$$

$$C13\ \frac{p_1 \sim_\sigma^{\sigma'} p_2}{\sigma \parallel p_1 = p_2 \rightsquigarrow \sigma' \parallel 1} \qquad C14\ \frac{p_1 \not\sim_\sigma^{\sigma'} p_2}{\sigma \parallel p_1 = p_2 \rightsquigarrow \sigma \parallel 0}$$

$$C15\ \frac{x \in \mathit{fresh};\ s \in \{0,1\}}{\sigma \parallel ?_s v \rightsquigarrow \sigma \parallel ?_s(x, \varnothing \parallel x \in v)_R} \qquad C16\ \frac{\theta \rightsquigarrow \theta'}{\sigma \parallel ?_s(x,\theta)_R \rightsquigarrow \sigma \parallel ?_s(x,\theta')_R}$$

$$C17\ \frac{}{\sigma \parallel ?_s(x, \bigvee(\langle\sigma' \parallel 1\rangle \frown \overline{\theta}))_R \rightsquigarrow \sigma \parallel s} \qquad C18\ \frac{}{\sigma \parallel ?_s(x,\sigma' \parallel 0)_R \rightsquigarrow \sigma \parallel \mathit{cpl}\ s}$$

Rules $C15$ to $C18$ handle the resolution of tests of emptiness and non-emptiness of sets. Similar to the treatment of μ in expression reduction, a sub-resolution is started which enumerates the elements of the set v by resolving the constraint $x \in v$. The state of this resolution is stored in an intermediate property expression, $?_s(x,\theta)_R$, where s is either 1 or 0, depending on the pole of the test. The test succeeds (resp. fails, depending on s) if at least one constraint can be solved, it fails (resp. succeeds) if no constraint can be solved (which requires to enumerate the entire set).

4 Implementation

The computation model of μZ implies the design of an abstract machine, called ZAM, whose implementation in C++ is currently in progress. The ZAM is a *concurrent* constraint resolution machine, which has some similarities with the machine described for Oz in [11]. The experimental state of the design of the ZAM is described below.

Values Fig. 3 shows the basic types used by the ZAM to represent values. There are four variants of values: constructed terms, variables (represented by an index), sets, and native values, such as numbers.

Fig. 3 Value Types of the ZAM

```
typedef ValueData  const * Value;   typedef IntenData const * Inten;
typedef ConstrData const * Constr;
enum Kind          { TERM, VAR, SET, NATIVE };
struct ValueData   { Kind kind; };
struct TermData    : ValueData { Constr cons; Value args[]; };
struct VarData     : ValueData { int index; };
struct SetData     : ValueData { vector<Value> extens;
                                 vector<Inten> intens; };
struct NativeData  : ValueData { virtual int compare(...) = 0; ... };
struct IntenData   { int varcount; Value target; Value env[];
                     Constr constrs[]; };
struct ConstrData  { virtual Thread spawn(..., int shift); }
```

A set value, `SetData(extens,intens)`, consists of two sorted sequences, one containing the *extensional* part and one containing the *intensional* part. Let v_j be the values in `extens`, and i_k be the intensions in `intens`, then a set value of the ZAM represents the μZ value $\{v_1\} \cup \ldots \cup \{v_n\} \cup i_1 \cup \ldots \cup i_m$. This representation is well suited for implementing union and intersection of sets. For union, we just merge in linear time the extension and intension sequences of both sets. For intersection of two sets, `SetData(extens1,intens1)` and `SetData(extens2,intens2)`, we build the set (in a pseudo notation)

```
SetData(std::set_intersection(extens1,extens2),
        {make_inten(extens1,intens2),
         make_inten(extens2,intens1),
         make_inten(intens1,intens2)})
```

Here, `make_inten(a,b)` constructs a new intension which conceptually executes $[\![x \mid x \in a \wedge x \in b]\!]$. Optimizations are possible if extensions and/or intensions of intersected sets are empty or singletons.

An intension (Fig. 3), describes the number of variables it allocates for its execution, its target value (representing the pattern in an μZ intension $[\![p \mid \theta]\!]$, a pointer to an array of values representing the *environment* the intension was constructed within, and an array of constraints. A constraint is specified by an abstract class, which just provides a method to *spawn* a *thread* executing this constraint (threads are explained later on). This abstraction allows the easy integration of specialized constraint implementations (for example, the intensions dynamically created with `make_inten` use constraints which have a non-standard way to spawn a thread).

When an intension is instantiated in the context of a goal, a block of variables is allocated in the variable table of the goal, and for each constraint a thread is spawn. The `shift` parameter of the spawn method defines the base index in the variable table for this instance of the intension. A thread obtains the actual index of a variable by adding to `shift` a constant offset.

Fig. 4 Control Types of the ZAM

```
typedef ThreadData * Thread; typedef GoalData * Goal;
struct ThreadData  { enum Status { NORMAL, WAIT, MWAIT, TWAIT } status;
                     Instr * pc; int shift; int dumpDepth;
                     enum Priority { LO, HI } prio;
                     Value env[]; Value registers[];
                     union { <<further data if in MWAIT or TWAIT>> }; };
struct VarInfo     { Value binding; vector<Thread> waiting; };
struct ChoiceInfo  { Thread initiator; <<data describing alternative>>;
                     vector<int> bound; vector<ThreaDump> dumps;
                     vector<Thread> enriched; };
struct ThreadDump  { Thread dumper; Instr * pc; };
struct GoalData    { Thread parent; vector<Thread> running; Value target;
                     vector<VarInfo>    varTable;
                     vector<ChoiceInfo> choiceTable; };
```

The environment, `env`, of an intension contains bindings for variables which are free in the intension's constraints, and which are accessed by indexing from a constraint's thread. μZ's computation model demands that before an intension can be constructed, all such context variables are bound (cf. expression reduction rule $E3$), allowing the environment to be initialized at creation time of an intension. The situation is slightly different for fixed-point construction (cf. rule $E4$): during creation of an intension, the environment may contain holes, which are fixed after the intension has been created.

Threads and Goals A thread (Fig. 4) executes the instructions from the instruction stream pointed to by the program counter `pc`. A goal is a collection of threads which together work on a set of variables, described by the field `varTable`, and a stack of choice points, described by the field `choiceTable`.

A thread may be in one of several states, where the status `MWAIT` correspond to the rules $E15$-$E17$ for executing the μ-operator, and the status `TWAIT` to rules $C15$-$C18$ for executing the test for emptiness/non-emptiness of sets. In theses states, a thread executes a *subgoal*, and is suspended until the subgoal has finished. Subgoal execution will not be detailed in this paper.

In status `NORMAL`, the thread is keen on executing its next instruction; in status `WAIT` it waits for the binding of a variable. A simple round-robin scheduler with exactly two priority levels selects the next thread to be executed in status `NORMAL`. The priorities are used for realizing the Andorra principal ([19]) which is central for the efficiency of the ZAM: if a thread needs to create a choice point, it lowers its priority, giving any threads which can deterministically continue under the current variable binding preference. Thus choice point creation is dynamically postponed as long as possible.

Assuming choice point scheduling, a rather simple model for dumping can be used, which does not requires to track data dependencies. Whenever a thread executes an instruction and has not yet dumped its state to the current most inner choice, it does so. The field `dumpDepth` contains the maxi-

Fig. 5 Instruction Set of the ZAM

EMPTY		$\rightarrow r\|v$	MOVE	$r\|v\|e$	$\rightarrow r\|v$
SINGLE	$r\|v\|e$	$\rightarrow r\|v$	UNIFY	$r\|v\|e$, $r\|v\|e$	
INTEN	*inten* $r[]$	$\rightarrow r\|v$	MEMBER	$r\|v\|e$, $r\|v\|e$	
FIX	$r\|v$		MU	$r\|v\|e$	$\rightarrow r\|v$
TERM	*cons*, $r[]$	$\rightarrow r\|v$	TEST	$r\|v\|e$	
UNION	$r\|v\|e$, $r\|v\|e$	$\rightarrow r\|v$	TESTN	$r\|v\|e$	
ISECT	$r\|v\|e$, $r\|v\|e$	$\rightarrow r\|v$	SUCCESS		
WAIT	v				

mal choice depth a thread has been dumped to, and monotonically increases during thread's progress. The test for the need to dump is simply coded as `goal->choiceTable.size() > thread->dumpDepth`. This strategy would be hopeless inefficient if choice point scheduling would not be performed, since then threads would be backtracked arbitrarily, even if they do not depend on certain choice points. With choice point scheduling, however, independent computations are performed automatically *before* the choice point is created.

A choice point itself describes the thread which initiated the choice, some data which describes how to select the next alternative on backtrack, and trailing information. Trailing takes place for any actions performed during the time this choice is the innermost one: the field `bound` describes the set of variables which have been bound, field `dumps` saves the program counters of threads which made a step, and field `enriched` contains those threads which have been dynamically added (dynamic addition of threads appears when membership in an intension is executed; cf. resolution rule $C11$). On backtrack, the variable bindings are released, the dumped thread's pc is restored, and the enriched threads are killed.

It is sufficient to just restore the program counter to backtrack threads since the ZAM uses a "single-assignment" register model for storing temporaries (field `ThreadData::registers`): an assignment to a register is never overwritten. Thus a thread can be restored to an arbitrary execution point, always finding a valid temporary state at this point[3].

Instruction Set The ZAM uses a register transfer instruction language. The instructions are summarized in Fig. 5. The operands of instructions are the followings:

- r: the index of a register. This addresses the field `ThreadData::registers` of the executing thread.
- v: the index of a variable. This addresses the `GoalData::varTable` field of the goal the executing thread belongs to. If the variable is bound, its binding is denoted, otherwise a `VarData` term holding the variable's index. The executing thread adds the `ThreadData::shift` value to the index, which determines the base of the variable frame of on associated instantiation. If

[3] By construction, a thread can never dump in state `MWAIT` or `TWAIT`; thus special temporary control data stored in `ThreadData` for theses states needs not to be saved as well.

a variable is the destination of an instruction, then the value of the instruction's result is unified with the addressed variable's value.
- e: the index of an environment variable. This addresses the field `ThreadData::env` of the executing thread.
- $r|v|e$: alternatively, a register, variable, or environment index. Similar as the Java Virtual Machine, the ZAM provides in fact a set of specialized instructions for the different addressing modes, which avoid dynamic encoding at runtime, but from which is abstracted here.
- $r[]$: an "array" of registers. Provided is the starting index of a consecutive region of registers.
- *inten*: the index of an intension. This points to a global table of intensions belonging to the executed unit (the "constant pool" in notions of the JVM).
- *cons*: the index of a constructor. This points to a global table of constructors belonging to the executed unit.

The following remarks on the instructions themselves:

- The EMPTY instruction stores the empty set in the destination operand, the SINGLE instruction a singleton set containing the value in the source operand. The value in the source operand must not contain free variables, otherwise the behavior is undefined (the compiled code ensures this with the WAIT instruction).
- The INTEN instruction creates a new intension. The environment of the intension is found in the register array starting at $r[]$. In the intension's field `IntenData::env`, just a pointer to the register array is stored. The FIX instruction commits the environment of the addressed intension to be completely initialized. This instruction may make a copy of `IntenData::env`; however, by the assertion of "single assignments" to registers, the FIX instruction may as well do just nothing. (Making a copy avoids space leaks, since the live-time of the created intension may exceed the live-time of the creating thread; in this case, if no copy is made, the entire register array of the thread cannot be reclaimed by the garbage collector unless the intension is.)
- The TERM instruction creates a new constructor term. The arguments of the constructor are passed in the register array $r[]$. Again, a copy of this array is not required, though omitting it can create space leaks. (To copy or not, or to let this be determined by compiler optimizations, is an open question.)
- The UNION and ISECT instruction implement set union and intersection as described above. The source values must not contain free variables.
- The WAIT instruction lets the executing thread suspend until the addressed variable (and all the variables it indirectly refers to by a possible binding) are bound.
- The MOVE instruction transfers the source to the destination. If the destination is a variable and is bound, this is similar to unification. The UNIFY instruction performs a unification of its operands. If unification fails, the executing thread fails.

- The MEMBER instruction tests whether the first operand is member of the second set operand. The set operand must not contain free variables. This instruction is the only one which may create choice points: if the set contains more then one element, a choice point is pushed to the associated goal's choice stack, and execution continues with selecting the next element from the set. If this element is from the extension of the set, it is simply unified with the first operand of MEMBER. If it is an intension, then the current goal is enriched by threads executing an instance of this intension, as conceptually described by rule $C11$.
- The MU, TEST, and TESTN instructions create subgoals for resolving their operand, as specified by rules $E15$-$E17$ and rules $C15$-$C18$.
- The SUCCESS instruction terminates the executing thread. If all threads of a goal have been successfully terminated, then an associated parent thread of the goal is resumed (which is present if the goal is a subgoal).

Memory Management A Boehm-style conservative garbage collector [1] is employed for memory retrieval. The collector needs to handle inferior pointers (references "into" a memory cell), because pointers to register regions may be stored in values, but also since the standard template library of C++ is employed, which may use local allocation strategies for containers. Nevertheless, the implementation of the garbage collector is compact (300 lines of code) and portable: this becomes possible since garbage collection can be invoked synchronously in-between the execution steps of threads, where no stack exists, and the only root to consider is the machine configuration which points to the topmost goal.

Performance Since the compiler from μZ to ZAM code is not yet ready, performance measurements can be only very rudimentary. As a first guess, an "assembler" implementation of the list concatenation function *app* has been used:

$$\mathbf{fix}\, app \triangleleft \left[(\langle\rangle, ys, zs) \mid zs = ys \right] \cup \left[(x :: xs, ys, zs) \setminus t \mid (xs, ys, t) \in app \cap zs = x :: t \right]$$

All solutions to $app(xs, ys, c)$, where c is a constant list of length 128, have been queried in a loop executed thousand times (such that garbage collection needs to be invoked). The current implementation requires 16.4 seconds on a Pentium II/400, hence 16.4ms for one evaluation of *app*, backtracking over all possible 128 partitions. Around 2.9 million instruction steps are executed, which amounts to 177000 steps per second.

As a further test, a recursive definition of the *fac* function has been coded:

$$\mathbf{fix}\, fac \triangleleft \left[(x, y) \;\middle|\; x = 0 \cap y = 1 \right] \cup \left[(x, y) \;\middle|\; x > 1 \cap y = x * \mu((fac \cap \{x-1\}(\!|x \mapsto (x, _)|\!))(\!|(_, y) \mapsto y|\!)) \right]$$

For numbers and arithmetics, a native implementation is used (the ZAM has also some instructions for natives). Executing $fac(8, y)$ thousand times is done in 1.6 seconds, requiring 303000 steps: thus one recursive function application as implemented with the μ-operator requires around 0.2ms. In contrast, a highly efficient implementation of a functional language such as that of Opal [14] requires

around 0.0014ms per recursive call – 142 times faster. This is not suprising, since the code the Opal compiler generates for *fac* is identical regarding efficiency to that of an according recursive C function – whereas function application via the μ-operator deals with relations. Nevertheless, more effort has to be invested to get function application more efficient for special cases.

Taking into account that the ZAM is a virtual byte-code machine, these results are encouraging, though real benchmarking is required to get a clearer picture, and comparisons with the performance of other implementations of functional logic languages have to be made.

5 Conclusion

We have presented the μZ calculus, a strict computation model for it, and a sketch of the implementation of this model by the abstract machine ZAM.

Calculus. μZ provides a way on viewing the λ-calculus, predicate calculus, schema's calculus, set-algebra, etc., in a unified, small expression language. Though the motivation of μZ and its implementation is the execution of a subset of the specification language Z, the approach is also interesting for the implementation of declarative programming languages in general, since μZ can embed these languages. In its intention, the μZ calculus is comparable to the calculi given for Oz in [15], though it doesn't provides the "impure" concepts of memory cells and committed choice.

Computation Model. μZ's computation model is comparable to the one described in [6]. μZ's set values, $\bigcup\{\overline{v}\}\cup\bigcup[\![\overline{p} \mid \overline{\theta}]\!]$, correspond to the "definitional trees" used in this work. Higher-orderness has perhaps a cleaner representation in μZ, since set-values are first-order inhabitants of the operational domain which definitional trees are not – on the other hand, μZ's set values are more restricted then definitional trees since they only support the "or" and the "branch" constructor.

A strict computation order has been chosen for a easier traceability of execution by humans, which is important for the application to test-data evaluation, and a supposed better overall performance. It is drawn from experiences with functional programming and language implementation that the overhead of laziness has to be paid for significantly, which in practice often results in that programmers use strictness annotations [8].

Machine. The ZAM's principle design is a straight consequence of μZ's computation model. Some aspects of the design and implementation are comparable to the AMOZ machine for Oz described in [11]. Yet, whereas in this machine by default sequential execution takes place, and only instructions which need to suspend are scheduled for concurrent execution, in the ZAM threads are incorporated from the beginning of a resolution. This allows for the simple but very effective optimization of dynamic choice scheduling.

The use of C++ as the implementation language of the ZAM is a consequence of experiences with an earlier implementation of a deterministic machine for μZ in Java (which uses concurrent constraint resolution for dealing with "parallel and", but cannot backtrack). This implementation directly compiles to Java instead of interpreting virtual code. Java's thread system is employed for executing

concurrent constraints. The approach turned out to be problematic for several reasons. First, an order of magnitude of performance is lost because of the interpretative character of current Java implementations themselves – meanwhile the advantage of portability does not count so much for an abstract machine implementation which does not rely on library functionality (a core of C++ is supported on all important platforms today). Second, the "clean" language design of Java hinders some "tricks" one would like to apply for the sake of efficiency in an implementation: allocating unitialized memory, pointer calculation, working with double indirections, and so on. Third, Java threads, which are the only (efficient) possibility to implement context switching in an non-interpretative approach, turned out to have serious restrictions on some platforms: there may be an upper bound of threads which can be allocated at the same time, and there is no guarantee about the allocation costs, since the thread model of Java is declared to be "medium-weighted" in some implementations (for example the JDK port to Linux).

Animating Z. Some basics about a mapping from Z to μZ have been shown, but we have not discussed the intrinsic details, which are out of the scope of this paper. In reality, a mapping of full Z has to convert axioms to definitions where possible, e.g. eliminate quantifiers in axioms of the kind $\forall\, x : X \bullet f(x) = e$, map recursion to μZ's fixed-point operator, and more. The mapping also has to weaken the specification by turning some axioms into assumptions: if a user e.g. declares a function to be "total" by using Z's total function arrow, $f : A \longrightarrow B$, this assertion cannot be computed if f's definition is non-finite, and need therefore be converted into an assumption. All of this has been implemented in the ZAP tool [4], and should port to the new design of the computation model.

There are several other approaches for the animation of Z. Executing the "imperative" part of Z is provided by the ZANS tool [9], where imperative means the execution of Z's specification style for sequential systems using state transition schemas. A translation to Haskell is described in [3], where monads are used for dealing with the sequential specification style, but no logic resolution is employed. Translations to Prolog are described e.g. in [20]. The approach presented in this paper goes beyond all these, since it allows for the combination of the functional and logic aspects of Z in a higher-order setting – getting the sequential specification style as a trade-off. Related to our work is the translation of Z to Mercury [21]: however, in this approach the effective power of execution is restricted in comparison to ours, since the flow of control has to be determined statically.

References

1. H. Boehm. Space efficient conservative garbage collection. In *Proceedings of the ACM SIGPLAN '93 Conference on Programming Language Design and Implementation*, volume 28 of *SIGPLAN Notices*, pages 197–206, June 1993.
2. R. Büssow and W. Grieskamp. A Modular Framework for the Integration of Heterogenous Notations and Tools. In K. Araki, A. Galloway, and K. Taguchi, editors, *Proc. of the 1st Intl. Conference on Integrated Formal Methods – IFM'99*. Springer-Verlag, London, June 1999.

3. H. S. Goodman. The Z-into-Haskell tool-kit: An illustrative case study. In J. P. Bowen and M. G. Hinchey, editors, *ZUM'95: The Z Formal Specification Notation*, volume 967 of *Lecture Notes in Computer Science*, pages 374–388. Springer-Verlag, 1995.
4. W. Grieskamp. *ZAP User's Guide*. Technische Universität Berlin, Dec. 1998. The ZETA System Documentation, URL: `http://uebb.cs.tu-berlin.de/zeta`.
5. M. Hanus. The integration of functions into logic programming: From theory to practice. *Journal of Logic Programming*, 19(20), 1994.
6. M. Hanus. A unified computation model for declarative programming. In *Proc. of the 1997 Joint Conference on Declarative Programming*, Grado (Italy), 1997.
7. M. Hanus. Curry – an integrated functional logic language. Technical report, Internet, 1999. Language report version 0.5.
8. P. H. Hartel, M. Feeley, M. Alt, L. Augustsson, P. Baumann, M. Beemster, E. Chailloux, C. H. Flood, W. Grieskamp, J. H. G. van Groningen, K. Hammond, B. Hausman, M. Y. Ivory, R. E. Jones, J. Kamperman, P. Lee, X. Leroy, R. D. Lins, S. Loosemore, N. Röjemo, M. Serrano, J.-P. Talpin, J. Thackray, S. Thomas, P. Walters, P. Weis, and P. Wentworth. Benchmarking implementations of functional languages with "pseudoknot", a Float-Intensive benchmark. *J. of Functional Programming*, 6(4), 1996.
9. X. Jia. An approach to animating Z specifications. Internet: `http://saturn.cs.depaul.edu/~fm/zans.html`, 1996.
10. G. Kahn. Natural semantics. In *Symposium on Theoretical Computer Science (STACS'97)*, volume 247 of *Lecture Notes in Computer Science*, 1987.
11. M. Mehl, R. Scheidhauer, and C. Schulte. An Abstract Machine for Oz. Research Report RR-95-08, Deutsches Forschungszentrum für Künstliche Intelligenz, Stuhlsatzenhausweg 3, D66123 Saarbrücken, Germany, June 1995. Also in: *Proceedings of PLILP'95*, Springer-Verlag, LNCS, Utrecht, The Netherlands.
12. K. Moon. *Implementation of Subset Logic Languages*. PhD thesis, State University of New York at New Buffalo, 1997.
13. O. Owe. Partial logic reconsidered: A conservative approach. *Formal Aspects of Computing*, 5:208–223, 1997.
14. W. Schulte and W. Grieskamp. Generating Efficient Portable Code for a Strict Applicative Language. In *Phoenix Seminar and Workshop on Declarative Programming*. Springer Verlag, Berlin, Heidelberg, New York, 1992.
15. G. Smolka. A calculus for higher-order concurrent constraint programming with deep guards. Research Report RR-94-03, Deutsches Forschungszentrum für Künstliche Intelligenz, Stuhlsatzenhausweg 3, D-66123 Saarbrücken, Germany, Feb. 1994.
16. J. M. Spivey. *The Z Notation: A Reference Manual*. Prentice Hall International Series in Computer Science, 2nd edition, 1992.
17. F. Stolzenburg. An algorithm for general set unification and its complexity. *Journal of Automated Reasoning*, 22(1), 1999.
18. R. L. Vaught. *Set Theory – An Introduction*. Birkhäuser, 1995.
19. D. H. D. Warren. The extended andorra model with implicit control. In *ICLP'90 Parallel Logic Programming Workshop*, 1990.
20. M. M. West and B. M. Eaglestone. Software development: Two approaches to animation of Z specifications using Prolog. *IEE/BCS Software Engineering Journal*, 7(4):264–276, July 1992.
21. M. Winikoff, P. Dart, and E. Kazmierczak. Rapid prototyping using formal specifications. In *Proceedings of the Australasian Computer Science Conference*, 1998.